```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m25.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m25.c -o m25
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m25
==========================================
       PIPE AND STREAM REDIRECTION
==========================================
74
Data flow completed successfully.
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    pid_t child1;
    pid_t child2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("=============================================\n");
    printf("       PIPE AND STREAM REDIRECTION\n");
    printf("=============================================\n");

    child1 = fork();

    if (child1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        /*
         * Child 1:
         * Execute "ls -l"
         * and redirect stdout to pipe.
         */

        close(pipefd[0]);

        if (dup2(pipefd[1], STDOUT_FILENO) == -1)
        {
            perror("dup2");
            exit(EXIT_FAILURE);
        }

        close(pipefd[1]);
```

```
    if (child2 == 0)
    {
        /*
         * Child 2:
         * Execute "wc -l"
         * and redirect stdin from pipe.
         */

        close(pipefd[1]);

        if (dup2(pipefd[0], STDIN_FILENO) == -1)
        {
            perror("dup2");
            exit(EXIT_FAILURE);
        }

        close(pipefd[0]);

        execlp(
            "wc",
            "wc",
            "-l",
            NULL
        );

        perror("execlp");
        exit(EXIT_FAILURE);
    }

    /*
     * Parent does not use the pipe.
     */
    close(pipefd[0]);
    close(pipefd[1]);

    /*
     * Wait for both children.
     */
    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Data flow completed successfully.\n");

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m26.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m26.c -o m26
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m26
==========================================
       MULTIPLE PIPELINE EXECUTION
==========================================

Examples:
ls -l | grep txt | wc -l
cat /etc/passwd | grep root | cut -d: -f1
ps aux | grep bash | wc -l

pipeline> -d

Commands in pipeline: 1
execvp: No such file or directory

Pipeline completed.

pipeline>
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 2048

typedef struct
{
    char *args[MAX_ARGS];
    int argc;
} Command;

void parse_command(char *text, Command *command)
{
    char *token;
    int i = 0;

    token = strtok(text, " \t\n");

    while (token != NULL &&
           i < MAX_ARGS - 1)
    {
        command->args[i++] = token;

        token = strtok(NULL, " \t\n");
    }

    command->args[i] = NULL;
    command->argc = i;
}

int parse_pipeline(char *input,
                   Command commands[])
{
    char *command_text;
    int count = 0;

    command_text = strtok(input, "|");
```

```
        token = strtok(NULL, " \t\n");
    }

    command->args[i] = NULL;
    command->argc = i;
}

int parse_pipeline(char *input,
                   Command commands[])
{
    char *command_text;
    int count = 0;

    command_text = strtok(input, "|");

    while (command_text != NULL &&
           count < MAX_COMMANDS)
    {
        parse_command(
            command_text,
            &commands[count]
        );

        if (commands[count].argc > 0)
            count++;

        command_text = strtok(NULL, "|");
    }

    return count;
}

void execute_pipeline(Command commands[],
                      int command_count)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    int i;
    int j;

    /*
     * Create all required pipes.
     */
    for (i = 0; i < command_count - 1; i++)
```

```
    int i;
    int j;

    /*
     * Create all required pipes.
     */
    for (i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            exit(EXIT_FAILURE);
        }
    }

    /*
     * Create processes.
     */
    for (i = 0; i < command_count; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (pids[i] == 0)
        {
            /*
             * Redirect stdin from previous pipe.
             */
            if (i > 0)
            {
                if (dup2(
                        pipes[i - 1][0],
                        STDIN_FILENO) == -1)
                {
                    perror("dup2 stdin");
                    exit(EXIT_FAILURE);
                }
            }

            /*
             * Redirect stdout to next pipe.
```

```
            }
        }

        /*
         * Redirect stdout to next pipe.
         */
        if (i < command_count - 1)
        {
            if (dup2(
                    pipes[i][1],
                    STDOUT_FILENO) == -1)
            {
                perror("dup2 stdout");
                exit(EXIT_FAILURE);
            }
        }

        /*
         * Close every pipe descriptor
         * after redirection.
         */
        for (j = 0;
             j < command_count - 1;
             j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        /*
         * Execute command.
         */
        execvp(
            commands[i].args[0],
            commands[i].args
        );

        perror("execvp");
        exit(EXIT_FAILURE);
    }
}

/*
 * Parent closes all pipe descriptors.
 */
for (i = 0;
     i < command_count - 1;
```

```
    /*
     * Parent closes all pipe descriptors.
     */
    for (i = 0;
         i < command_count - 1;
         i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /*
     * Wait for all child processes.
     */
    for (i = 0;
         i < command_count;
         i++)
    {
        waitpid(
            pids[i],
            NULL,
            0
        );
    }
}

int main()
{
    char input[MAX_INPUT];

    Command commands[MAX_COMMANDS];

    int command_count;

    printf("================================================\n");
    printf("       MULTIPLE PIPELINE EXECUTION\n");
    printf("================================================\n");

    printf("\nExamples:\n");
    printf("ls -l | grep txt | wc -l\n");
    printf("cat /etc/passwd | grep root | cut -d: -f1\n");
    printf("ps aux | grep bash | wc -l\n");

    while (1)
    {
        printf("\npipeline> ");
        fflush(stdout);
```

```
    while (1)
    {
        printf("\npipeline> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        command_count =
            parse_pipeline(
                input,
                commands
            );

        if (command_count == 0)
        {
            printf("Invalid pipeline.\n");
            continue;
        }

        if (command_count > MAX_COMMANDS)
        {
            printf("Too many commands.\n");
            continue;
        }

        printf("\nCommands in pipeline: %d\n",
               command_count);

        execute_pipeline(
            commands,
            command_count
        );
```

```
        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        command_count =
            parse_pipeline(
                input,
                commands
            );

        if (command_count == 0)
        {
            printf("Invalid pipeline.\n");
            continue;
        }

        if (command_count > MAX_COMMANDS)
        {
            printf("Too many commands.\n");
            continue;
        }

        printf("\nCommands in pipeline: %d\n",
               command_count);

        execute_pipeline(
            commands,
            command_count
        );

        printf("\nPipeline completed.\n");
    }
```